Flash Sale Checkout: Design Report

Candidate: Unnathi Vinod · **Role:** Full-Stack Software Engineer **Code:** https://github.com/unnathivinod/flash-sale-system

1. The problem in one paragraph

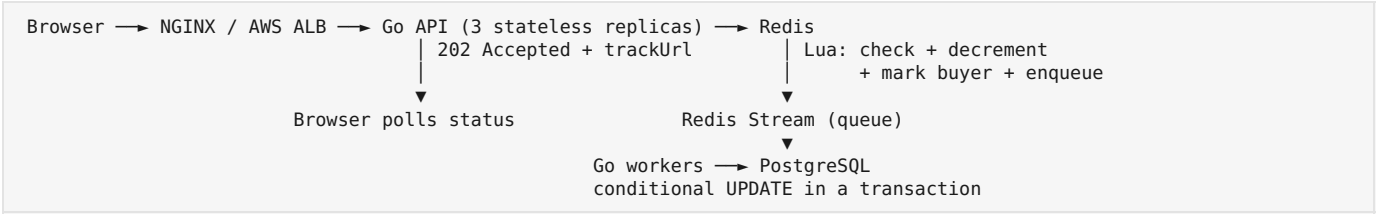
500 phones go on sale at 8:00 PM and more than 100,000 checkout requests arrive in the first 10 seconds. The old system read the stock, then updated it, inside every HTTP request. Under load this caused three failures: **timeouts** (every request waits on the database), **row-lock contention** (100,000 requests fight over the *same* inventory row), and **overselling** (540 orders for 500 phones, because two requests could read "1 left" at the same moment and both succeed).

2. Root cause analysis

Symptom	Root cause
Overselling (540 > 500)	*Check-then-act race.* `SELECT stock` and `UPDATE stock = <value>` are two separate steps. Between them, other requests read the same old value ("lost update").
Row lock timeouts	All requests update **one row** (`product_id = 1`). Postgres lets only one transaction hold that row's lock, so everyone else queues behind it and times out. This is a *hot row*.
High latency	The database write sits **inside the HTTP request**. Users wait for the slowest part of the system.
N+1 query pattern	Each request makes several separate DB round trips (read stock, update stock, insert order, plus reads of user/product). N requests × several queries each = hundreds of thousands of round trips in 10 seconds against one database.
Duplicate submissions	Users mash the button; nothing stops the same user from creating several orders.

Key insight: the database is the slowest and least scalable part, so it must not be on the hot path. We decide *who wins* in memory (Redis, atomically), and persist winners to the database *later*, at a pace it can handle.

3. Architecture



Layer	Choice	Why
Client	Plain HTML/JS, ~10 KB	Interactive almost instantly; nothing to download
Edge / gateway	NGINX locally, **AWS ALB** in the cloud	Load balancing, health checks, rate limiting, bot blocking
API	**Go**, stateless, 3 replicas	Goroutines handle thousands of concurrent connections cheaply; stateless means scale by adding copies
Inventory	**Redis + Lua script**	Single-threaded, so a script runs atomically; ~100k ops/s on one core
Queue	**Redis Streams** with a consumer group	Durable, acknowledged delivery and crash recovery; no extra system to run (swappable for Kafka/RabbitMQ/SQS)
Persistence	**PostgreSQL**	ACID transactions, constraints as the final safety net

4. Frontend strategy

Technique	What we did	Why it matters
Click lock (throttle)	Button disables on the first click; a `locked` flag ignores all later clicks	One user = one request, not ten
Idempotent retries	Server returns the *same* `orderId` for the same user, so retries are safe	Network blips don't create duplicate orders
Optimistic UI	Shows "You're in! Holding your spot…" instantly, before the server answers	No frozen screen; feels fast even when the server is busy
Exponential backoff + jitter	Retries after ~1s, 2s, 4s, 8s with ±20% randomness	Stops 100k clients retrying at the same instant ("thundering herd")
Polling, not WebSockets	Sale status every 1–3s (with jitter); order status with backoff	Stateless and cache-friendly; holding 100k open sockets would need sticky sessions and much more memory. **SSE** is the upgrade path if we need push.
Server-synced countdown	Countdown uses server time offset	Users with wrong clocks can't click early
Bundle size	Single 9.7 KB file (≈4 KB gzipped), inline CSS/JS/SVG, no framework	Target was < 200 KB; one request, parses in milliseconds, so TTI < 1.5s on slow phones
CDN / edge caching	Static files served by NGINX (not the API) with long cache headers; in production S3 + CloudFront	Static traffic never touches the API servers
Pre-fetching	The page loads sale data before the sale starts, so only the checkout call happens at 8 PM	Moves work out of the critical 10 seconds

5. Backend and middleware

Load balancing. NGINX uses `least_conn` across 3 API replicas, keeps connections alive, and retries another replica if one fails. On AWS, an Application Load Balancer spreads traffic across two EC2 instances in different Availability Zones and removes unhealthy ones using `/healthz`.

Bot and abuse protection. - Block empty or scripted User-Agents. - Rate limit per IP: 5 checkout requests/second (a human clicks once). Excess gets **429** with `Retry-After`, which the client handles gracefully. - On AWS, the same limiter runs inside the Go API with counters **in Redis**, so all replicas share one limit. In production: **AWS WAF** rate-based rules + bot control.

The hot path (`POST /api/checkout`). No database call. Steps: 1. Validate input (tiny body limit, strict `userId` format). 2. Run the Lua script. It first checks whether this user already has an order (returns the same one), then checks stock, decrements, records the buyer, and appends the order to the queue, **in one atomic step**. 3. Reply **202 Accepted** with `trackUrl`, or **409** if sold out.

Asynchronous workers. Workers read the Redis Stream as a consumer group (each message goes to exactly one worker) and write each order in its own transaction. A message is **acknowledged only after the database commit**. If a worker crashes mid-way, the message stays pending and another worker claims it after 30 seconds (`XAUTOCLAIM`). Nothing is lost.

Go event-loop safety. Every request runs in its own goroutine; nothing blocks on the database, and all shared state lives in Redis. The API keeps no in-memory state between requests, so there are no locks or global maps to get wrong.

6. Database consistency and concurrency control

Pessimistic vs optimistic vs atomic

Approach	How	Correct?	At 100k requests/10s
Naive `SELECT` then `UPDATE`	Read value, compute, write back	**No**: lost updates, oversells	Fast but wrong
Pessimistic `SELECT ... FOR UPDATE`	Lock the row, then update	Yes	Every request waits for one row lock → queueing, timeouts. Correct but slow.
Optimistic (version column)	`UPDATE ... WHERE version = v`; retry on conflict	Yes	Almost every request conflicts on the hot row → **retry storm**, wasted work
Atomic conditional `UPDATE ... WHERE stock > 0`	Check and write in one statement	Yes	Better, but still one hot row in the request path
Redis Lua token (our choice) + async DB	Decide winners in memory atomically; persist later	Yes	Microseconds per decision; DB sees only ~500 writes, not 100k

Why Lua in Redis is safer than SELECT-then-UPDATE

Redis executes commands on a single thread. A Lua script runs **from start to finish with nothing else in between**, so "check stock", "decrement", "record buyer", and "enqueue" behave as one indivisible step. There is no gap for another request to slip into, which is exactly the gap that caused 540 sales. It's also fast because it's in memory and needs one network round trip.

Defense in depth: 4 layers stop overselling

Layer	Mechanism	Protects against
1	Redis Lua token (atomic)	The race itself
2	`UPDATE inventory SET stock = stock - 1 WHERE product_id = $1 AND stock > 0`	Redis being wrong or out of sync
3	`CHECK (stock >= 0)` on the table	Any code path that forgets the condition
4	`UNIQUE (user_id, product_id)` + `order_id` primary key	Duplicate orders and messages delivered twice

Failure scenarios

What fails	What happens
Worker crashes mid-order	Message not acknowledged → reclaimed by another worker → processed once (order_id check)
Database briefly down	Workers keep retrying; users still get instant 202s; orders confirm when the DB returns
An API replica dies	Load balancer health check removes it; others continue (stateless)
Redis restarts	AOF persistence (fsync every second). On startup the API rebuilds the stock count from Postgres (the source of truth). DB constraints still prevent overselling. In production: ElastiCache with a replica and automatic failover
Same user clicks 10 times	Lua returns the same `orderId`; no new stock taken

7. Results

Measured with k6 on a Windows laptop (Docker Desktop): NGINX, 3 Go API replicas, 2 workers, Redis, Postgres.

Test	Concurrent users	Requests	Phones sold (stock 500)	Oversold	Dropped	p95 latency	Throughput
Naive design (SELECT then UPDATE)	300	3,000	**3,000**	**+2,500**	0%	1.59 s	318 req/s
Fixed design (Redis Lua + queue)	**1,000**	**20,000**	**500**	**0**	**0%**	**455 ms**	**1,862 req/s**

After the fixed run: **500 confirmed orders in Postgres, DB stock 0, Redis tokens 0, queue empty**. 19,500 users got a clean “sold out” (409); none got an error. 21 simulated double-clicks all returned the same order ID.

Takeaway: with more than 3× the concurrent users, the fixed design was ~6× higher throughput and 3.5× lower p95 latency, and it sold exactly the stock. The naive design oversold far more than the 540 seen in production, because without any lock nearly every request read the same stale stock value.

8. AWS production design

Component	Demo deployment	Full production
Entry	Application Load Balancer, 2 AZs	ALB + **AWS WAF** (rate + bot rules) + **CloudFront** for static files
API	2 × EC2 (Docker)	**ECS Fargate** service with auto scaling on request count
Inventory + queue	Redis on EC2	**ElastiCache for Redis** (Multi-AZ, auto failover)
Queue (alternative)	Redis Streams	**Amazon SQS** or MSK (Kafka) for higher durability needs
Database	Postgres on EC2	**RDS PostgreSQL** Multi-AZ
Monitoring	stats page	CloudWatch metrics/alarms on 5xx, queue depth, latency

Pre-scaling: flash sales are scheduled, so we **scale out before 8 PM** (scheduled scaling) instead of waiting for auto scaling to react, and warm the load balancer.

9. What I’d add next

1. **Payment with a reservation timeout.** Hold the phone for 10 minutes; if unpaid, return the token to stock.
2. **Waiting room / virtual queue** in front of the sale for very large launches.
3. **Real auth** (users identified by session/JWT instead of a guest id) and CAPTCHA on suspicious traffic.
4. **Reconciliation job** that compares Redis reservations with Postgres orders and alerts on any mismatch.